Introduction to autoencoders

Joseph Chazalon
joseph.chazalon (at) lrde.epita.fr

2018-07-10
L3i seminar on Deep Learning

Autoencoders: a definition

According to G. Hinton* (pioneer in autoencoders):

“Autoencoders are a way to project data on a curved latent manifold.”

This competes with Principal Components Analysis,
which is efficient
but can only find a linear manifold from a given dataset.

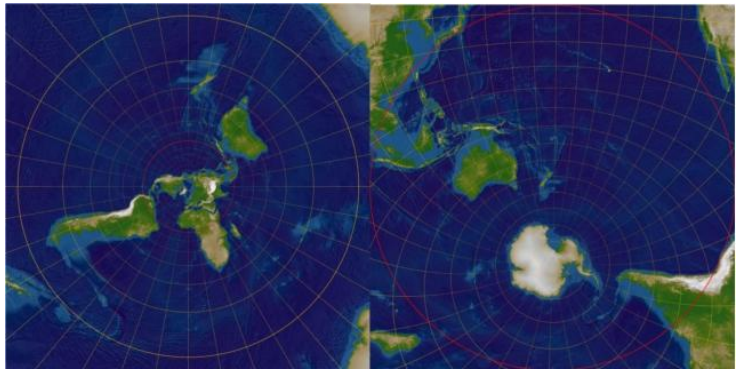
* https://www.coursera.org/lecture/neural-networks/from-pca-to-autoencoders-5-mins-JiT1i

A word about manifolds

According to Wikipedia contributors*:

“In mathematics, a **manifold** is a **topological space** that **locally resembles Euclidean space** near each point. More precisely, each point of an n-dimensional manifold has a neighbourhood that is homeomorphic to the Euclidean space of dimension n. In this more precise terminology, a manifold is referred to as an n-manifold.”

A 2-d manifold →



* https://en.wikipedia.org/wiki/Manifold

PCA vs autoencoders

PCA

Illustration on whiteboard

Autoencoders

Illustration on whiteboard

What are autoencoders actually good at?

According to F. Chollet* (creator of Keras, inventor of the Xception architecture):

“Today two interesting practical applications of autoencoders are **data denoising** [...] and **dimensionality reduction for data visualization**. With appropriate dimensionality and sparsity constraints, autoencoders can learn data projections that are more interesting than PCA or other basic techniques.”

“focusing on the reconstruction of a picture at the pixel level, for instance, is **not conductive to learning interesting, abstract features** of the kind that label-supervized learning induces [...]. One may argue that **the best features** in this regard are those that **are the worst at exact input reconstruction**”

* https://blog.keras.io/building-autoencoders-in-keras.html

Historical uses of autoencoders

Greedy layer by layer training of deep networks.

G. E. Hinton et R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks”, Science, vol. 313, nº 5786, p. 502-504, juill. 2006.

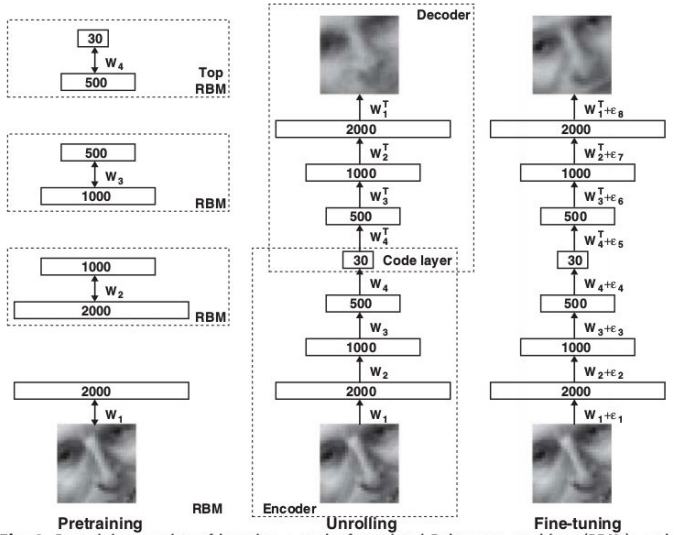


Fig. 1. Pretraining consists of learning a stack of restricted Boltzmann machines (RBMs), each having only one layer of feature detectors. The learned feature activations of one RBM are used as the “data” for training the next RBM in the stack. After the pretraining, the RBMs are “unrolled” to create a deep autoencoder, which is then fine-tuned using backpropagation of error derivatives.

D. Erhan, Y. Bengio, A. Courville, P.-A. Manzagol, P. Vincent, et S. Bengio, “Why does unsupervised pre-training help deep learning?”, Journal of Machine Learning Research, vol. 11, nº Feb, p. 625–660, 2010.

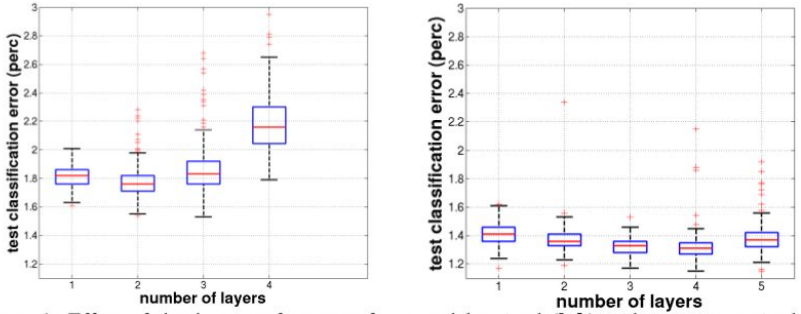


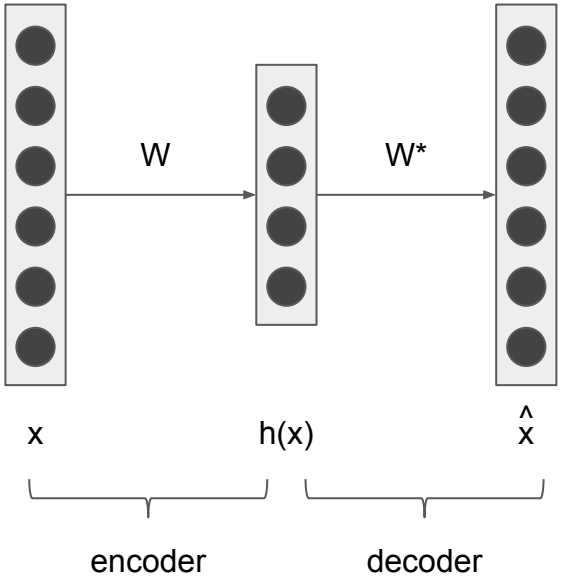
Figure 1: Effect of depth on performance for a model trained (**left**) without unsupervised pre-training and (**right**) with unsupervised pre-training, for 1 to 5 hidden layers (networks with 5 layers failed to converge to a solution, without the use of unsupervised pre-training). Experiments on MNIST. Box plots show the distribution of errors associated

It was before Batch Norm and Residual Learning.

Autoencoder architectures

Heavily inspired by H. Larochelle (warning: videos from 2013)
https://www.youtube.com/watch?v=FzS3tMl4Nsc&list=PL6Xpj9I5qXYEcOhn7TqghAJ6NAPrNmUBH&index=44

Linear dense autoencoder



h(x) = f(b + Wx)

x̂ = g(c + W*x)

f and g are linear functions.

W* should be the transpose of W on such simple network.

Reconstruction loss:

Cross entropy for binary data

$$-\sum_k (x_k \, log(\hat{x}_k)) + (1 - x_k) \, log(1 - \hat{x}_k))$$

Squash reconstructed values to [0,1] using a sigmoid activation

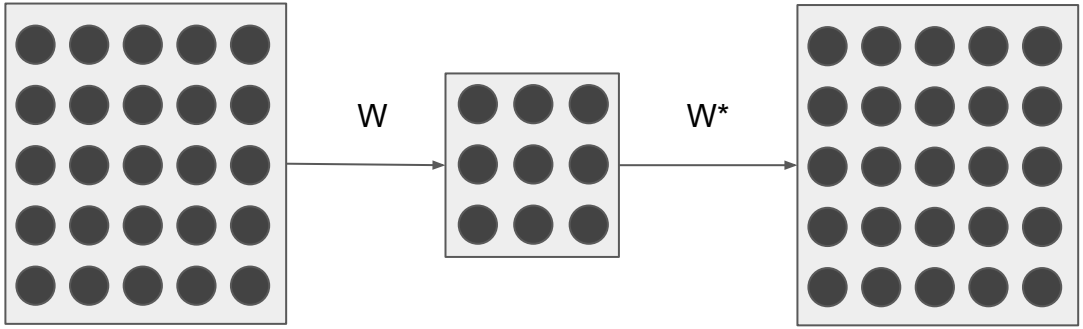
MSE for real-valued data

$$\frac{1}{2}\sum_k (\hat{x}_k - x_k)^2$$

Using linear activation + MSE loss, the encoder converges to the PCA decomposition (modulo a small transform) in a very slow way (but which scales to lots of data).

https://www.youtube.com/watch?v=xq-I0Rl8mt0&index=47&list=PL6Xpj9I5qXYEcOhn7TqghAJ6NAPrNmUBH

Convolutional autoencoder



Convolutions and deconvolutions as opposed to dense connexions

Better suited for images

Can be insensitive to input size

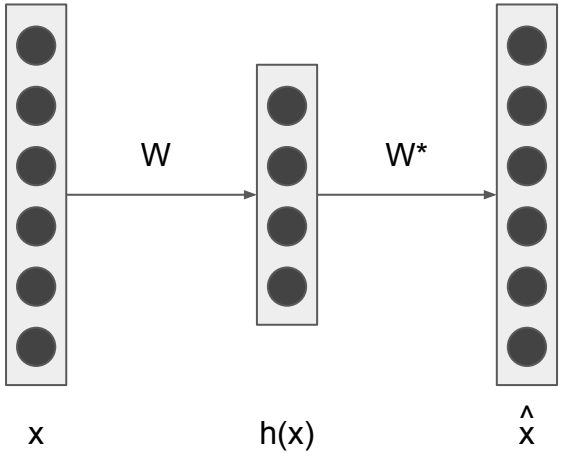
Can be used to learn filter banks

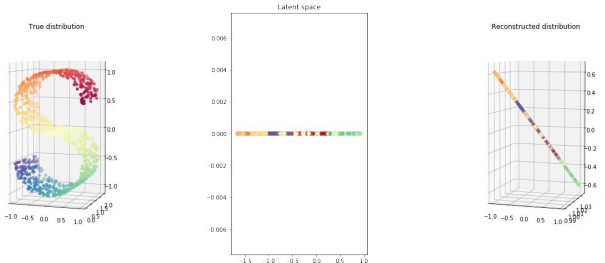
But are those features good for something except reconstruction?

Compression is good for a given domain, but generalizes badly.

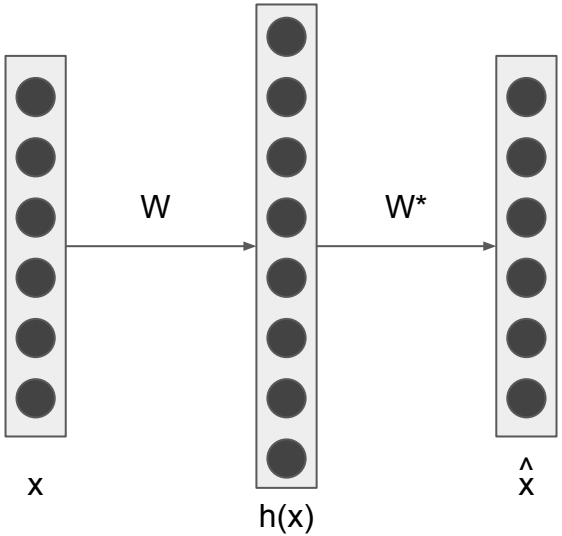
Undercomplete and overcomplete hidden layer

Undercomplete: lossy compression
Can collapse if not enough capacity.





Overcomplete: need regularization
Otherwise it will copy inputs to outputs



Use L1 regularization to generate linearly separable features (sparsity constraint) or...

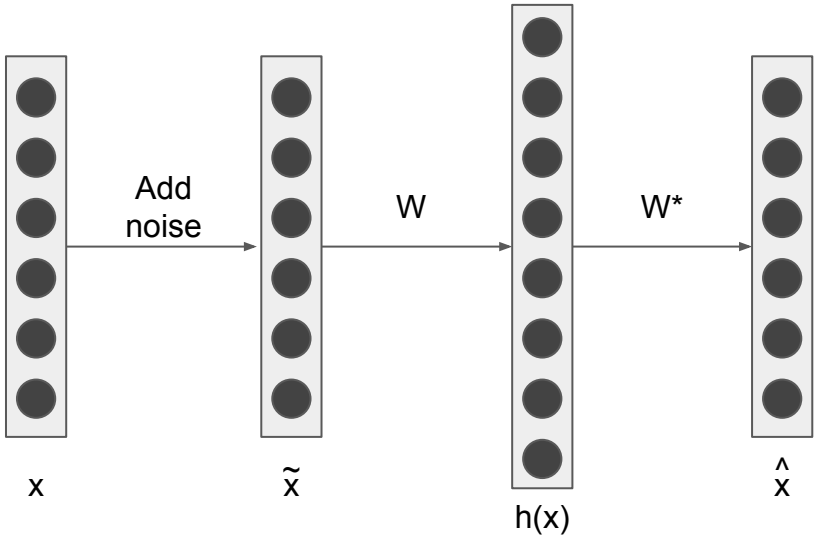
Denoising autoencoder

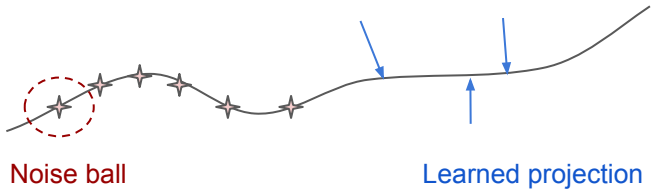
Denoising training

Randomly add noise to training samples:

- Set some values to 0
- Add gaussian noise
- …

Compare the **noiseless original input** with the reconstructed one.



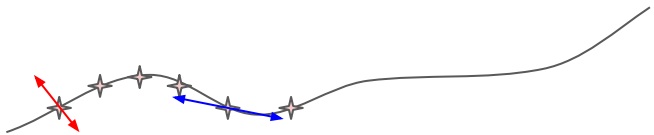


Contractive autoencoder

Add constraint on the encoder: its derivatives must be as small as possible around x_k.

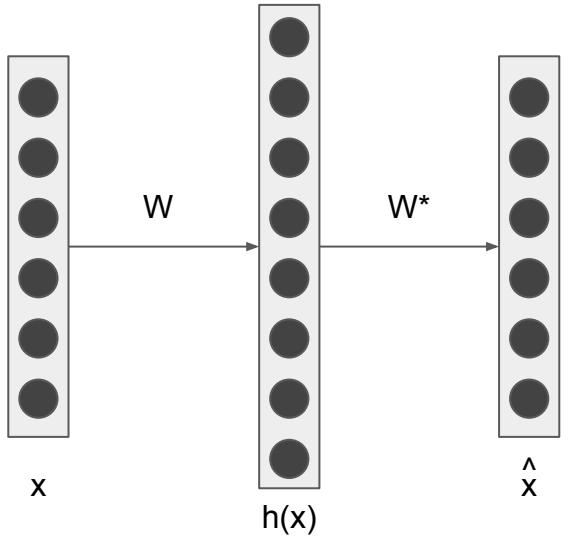
Both constraints (reconstruction + low variation) preserve gradients along the manifold and discard orthogonal ones.

Much like L1 regularization, but on the derivatives of the encoding function.



Useless gradient: discarded unit

Useful gradient: preserved unit



Can be overcomplete

Variational autoencoder

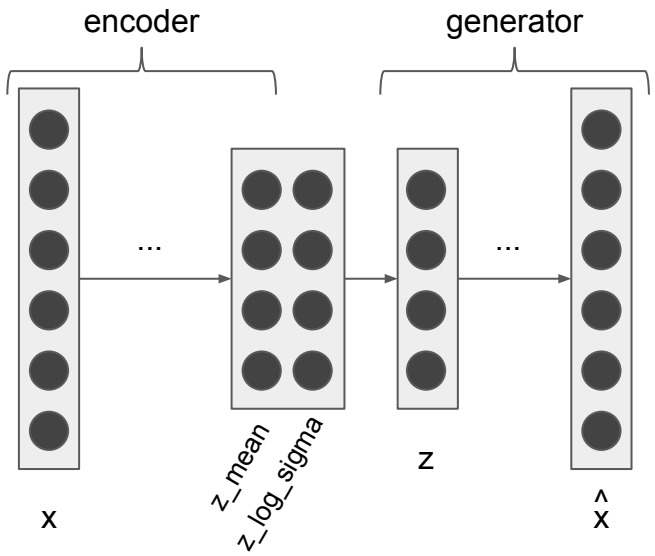
More modern, more generative power.

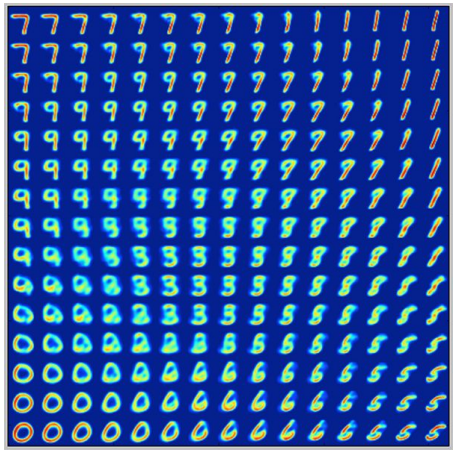
Constrain the latent space to match a sort of gaussian mixture.

Units in the central hidden layer are paired so that one encodes z_mean, and the other z_log_sigma.

Linear walks in the latent space now produce interesting transitions.

The loss is more complex (adds KL divergence constraint on z_mean & z_log_sigma).





In practice

In general, do not use simple autoencoders:

learn manifolds by retaining the bottleneck layer of a network

or use GANs → next talk

or use powerful visualization techniques like PCA, ZCA, ICA, LDA, T-SNE, UMAP